

Unit 2: Process Management

What is a Process?

- The program in execution is called a process.
- A program is an inanimate entity; only when a processor “breathes life” into it, does it becomes the “active” entity, we call a process.

Programs and Processes

- A process is different than a program.
- Consider the following analogy:

Scenario-1: A computer scientist is baking a birthday cake for his daughter

- Computer scientist - CPU
- Birthday cake recipe - program
- Ingredients - input data
- Activities: - processes
 - *reading the recipe*
 - *fetching the ingredients*
 - *baking the cake*

Programs and Processes...

- **Scenario-2: Scientist's son comes running in crying, saying he has been stung by a bee.**
 - Scientist records where he was in the recipe (the state of running process saved)
 - Reach first aid book and materials (another process fetched)
 - Follow the first aid action (high priority job)
 - On completion of aid, cake baking starts again from where it was left

A process is an activity of some kind, it has program, input, output and state.

TU Model Question

- What is the difference between Program and Process?

Process Models

- **Uniprogramming:** Only one process at a time.

Examples: Older systems

Advantages: Easier for OS designer

Disadvantages: Not convenient for user and poor performance

- **Multiprogramming:** Multiple processes at a time.

OS requirements for multiprogramming:

Policy: to determine which process is to schedule.

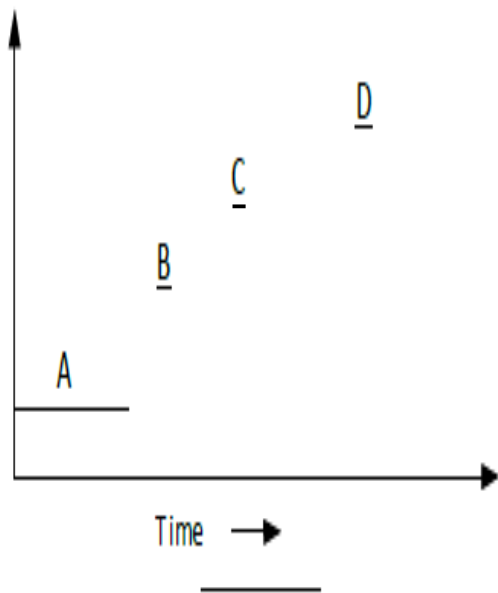
Mechanism: to switch between the processes.

Examples: Unix, Windows NT, etc

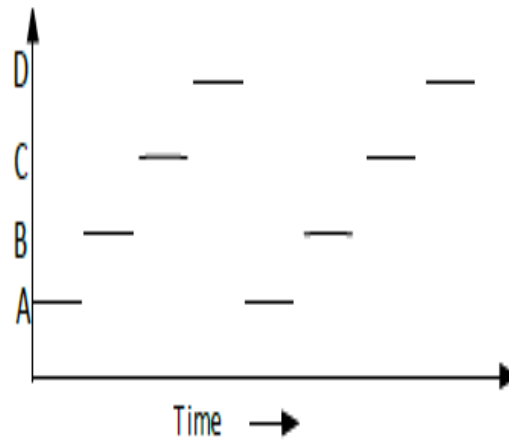
Advantages: Better system performance and user convenience.

Disadvantages: Complexity in OS

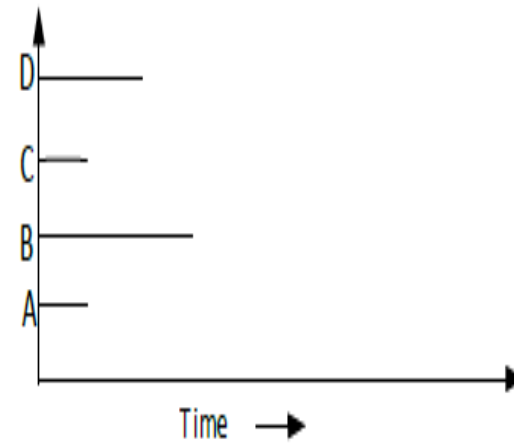
- **Multiprocessing:** System with multiple processors.



Uniprogramming



Multiprogramming



Multi processing

Process States

A process goes through a series of discrete process states.

- **Running state:**

- Process executing on CPU.
- Only one process at a time.

- **Ready state:**

- Process that is not allowed to CPU but is ready to run.
- A list of processes ordered based on priority.

- **Blocked state:**

- Process that is waiting for some event to happen.
(e. g. I/O completion events).
- A list of processes (no clear priority).

Process State Transitions

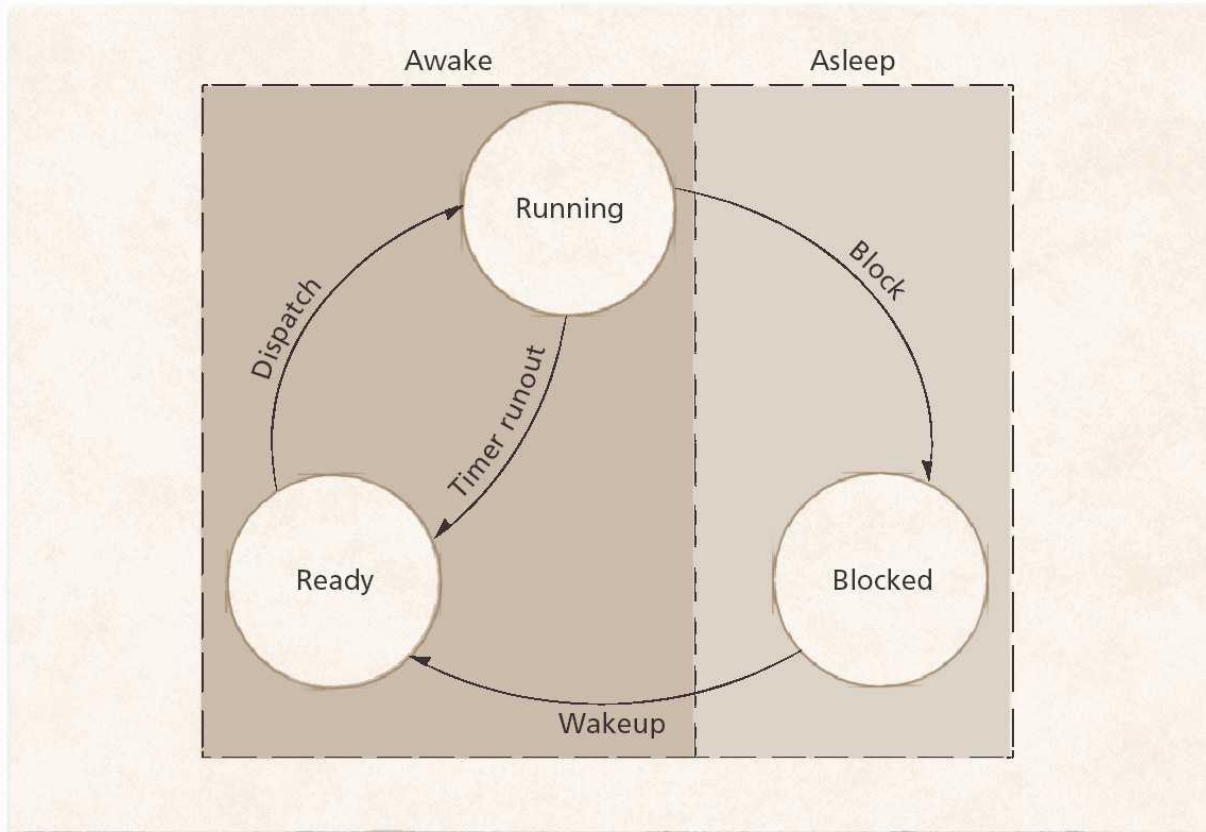


Figure: Process state transition diagram

Process State Transitions

- When a job is admitted to the system, a corresponding process is created and normally inserted at the back of the ready list.
- When the CPU becomes available, the process is said to make a state transition from ready to running.

dispatch(processname): ready -> running

- To prevent any one process from monopolizing the CPU, OS specifies a time period (quantum) for the process. When the quantum expires, the process makes state transition from running to ready.

timerrunout(processname): running -> ready

- When the process requires an I/O operation before quantum expires, the process voluntarily relinquishes the CPU and changes to the blocked state.

block(processname): running -> blocked

- When an I/O operation completes, the process makes transition from blocked state to ready state.

wakeup(processname): blocked -> ready

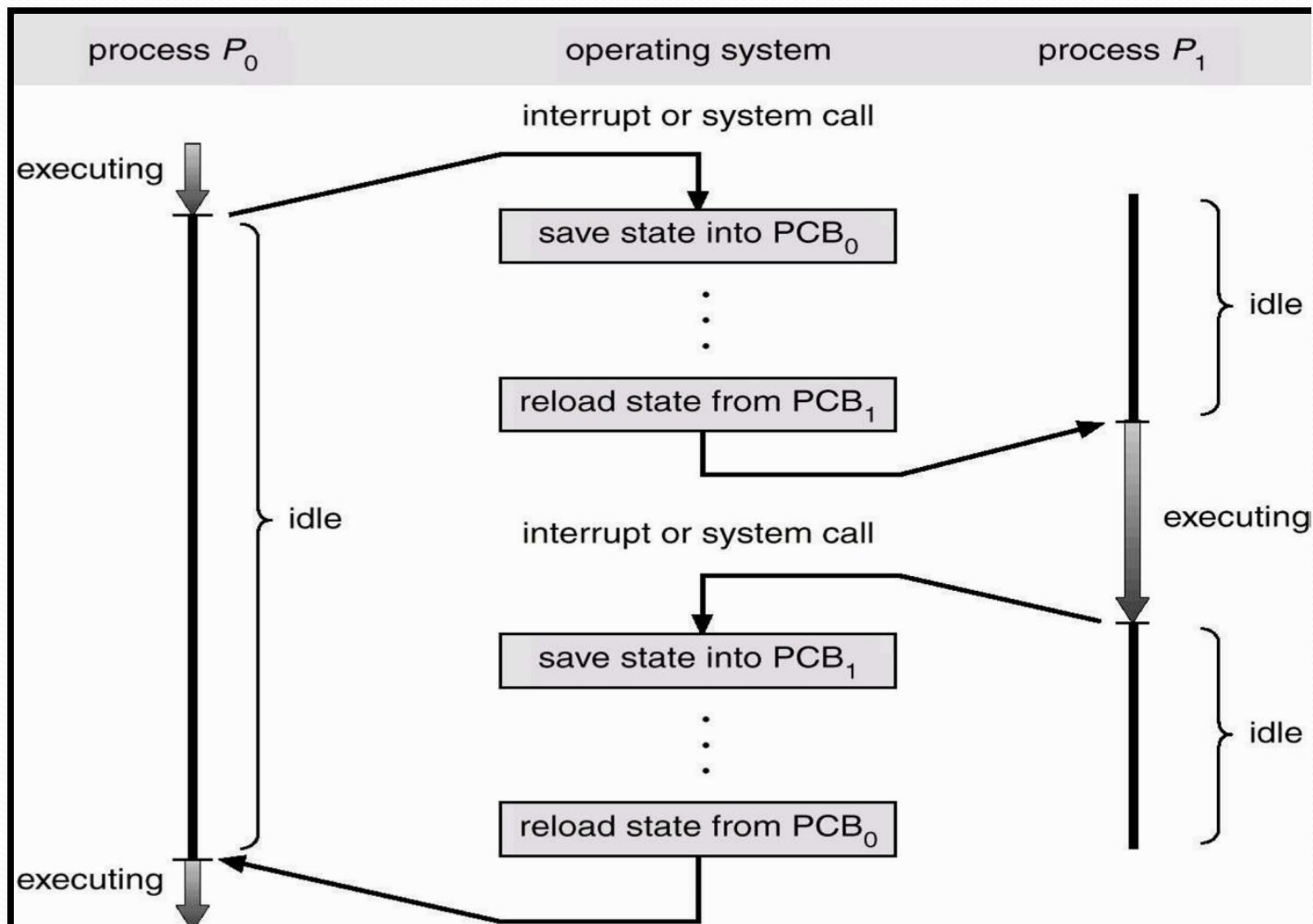
Implementation of Processes- Process Control Block

- To implement the process model, the operating system maintains a table (an array of structures), called the **process table (Process Control Block)**, with one entry per process. This entry contains information about the process' state, its program counter, stack pointer, memory allocation, the status of its open files, its accounting and scheduling information, and everything else about the process that must be saved when the process is switched from *running* to *ready* or *blocked* state so that it can be restarted later as if it had never been stopped.

PCB...

The PCB is the data structure containing certain important information about the process - also called process table or processor descriptor. The information included in it are:

- *Process state: Running, ready, blocked.*
- *Program counter: Address of next instruction for the process.*
- *Registers: Stack pointer, accumulator, PSW etc.*
- *Scheduling information: Process priority, pointer to scheduling queue, etc.*
- *Memory allocation: Value of base and limit register, page table, segment table, etc.*
- *Accounting information: time limit, process numbers etc.*
- *Status information: List of I/O devices, list of open files etc.*



Operations on Processes

- The processes in the system can execute concurrently, and they must be created and deleted dynamically.
- OS provides the mechanism for process ***creation*** and ***termination***.

Process Creation

There are four principal events that cause the process to be created:

- » *System initialization.*
- » *Execution of a process creation system call by a running process.*
- » *User request to create a new process.*
- » *Initiation of a batch job.*

- *When an operating system is booted, several processes are created. Some are foreground processes and others are background processes. For example, one background process may be designed to accept incoming email, sleeping most of the day but suddenly springing to life when email arrives. Processes that stay in the background to handle some activity such as email, Web pages, news, printing, and so on are called **daemons**.*

Process Creation...

- A running process may issue system calls to create several new processes during the course of execution. The creating process is called a **parent process**, whereas the new processes are called **children of that process**.
- In interactive systems, users can start a program by typing a command or (double) clicking an icon. Taking either of these actions starts a new process and runs the selected program in it.
- The last situation in which processes are created applies only to the batch systems found on large mainframes. Here users can submit batch jobs to the system (possibly remotely). When the operating system decides that it has the resources to run another job, it creates a new process and runs the next job from the input queue in it.

Process Termination

- After a process has been created, it starts running and does whatever its job is. Sooner or later the new process will terminate, usually due to one of the following conditions:
 1. Normal exit (voluntary).
 2. Error exit (voluntary).
 3. Fatal error (involuntary).
 4. Killed by another process (involuntary).
- Most processes terminate because they have done their work. When a compiler has compiled the program given to it, the compiler executes a system call to tell the operating system that it is finished. Screen-oriented programs also support voluntary termination. Word processors, Internet browsers and similar programs always have an icon or menu item that the user can click to tell the process to remove any temporary files it has open and then terminate.

Process Termination...

- The second reason for termination is that the process discovers a fatal error. For example, if a user types the command:

gcc foo.c

to compile the program *foo.c* and no such file exists, then the compiler simply exits. Screen-oriented interactive processes generally do not exit when given bad parameters. Instead they pop up a dialog box and ask the user to try again.

- The third reason for termination is an error caused by the process, often due to a program bug. Examples include executing an illegal instruction, referencing nonexistent memory, or dividing by zero.
- The fourth reason a process might terminate is that a process executes a system call telling the operating system to kill some other process. The *killer* must have the necessary authorization to do in the *killee*.

Assignment #2

1. What are disadvantages of too much multiprogramming?
2. List the definitions of process.
3. For each of the following transitions between the process states, indicate whether the transition is possible. If it is possible, give an example of one thing that would cause it.
 - a) Running -> Ready
 - b) Running -> Blocked
 - c) Blocked -> Running

TU Exam Question 2067

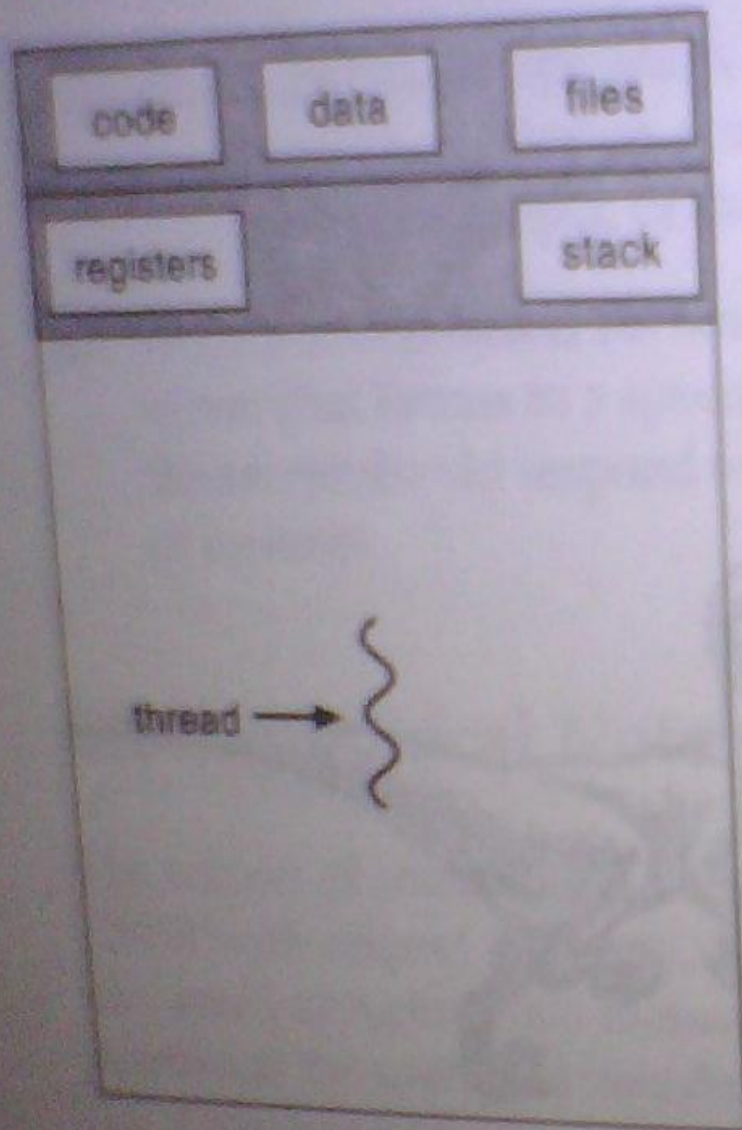
Does windows have any concept of process hierarchy? How does parent control the child?

- **Answer:** In some systems, when a process creates another process, the parent process and child process continue to be associated in certain ways. The child process can itself create more processes, forming a process hierarchy. Note that unlike plants and animals that use sexual reproduction, a process has only one parent (but zero, one, two, or more children). Windows does not have any concept of a process hierarchy. All processes are equal. The only place where there is something like a process hierarchy is that when a process is created, the parent is given a special token (called a handle) that it can use to control the child. However, it is free to pass this token to some other process, thus invalidating the hierarchy.

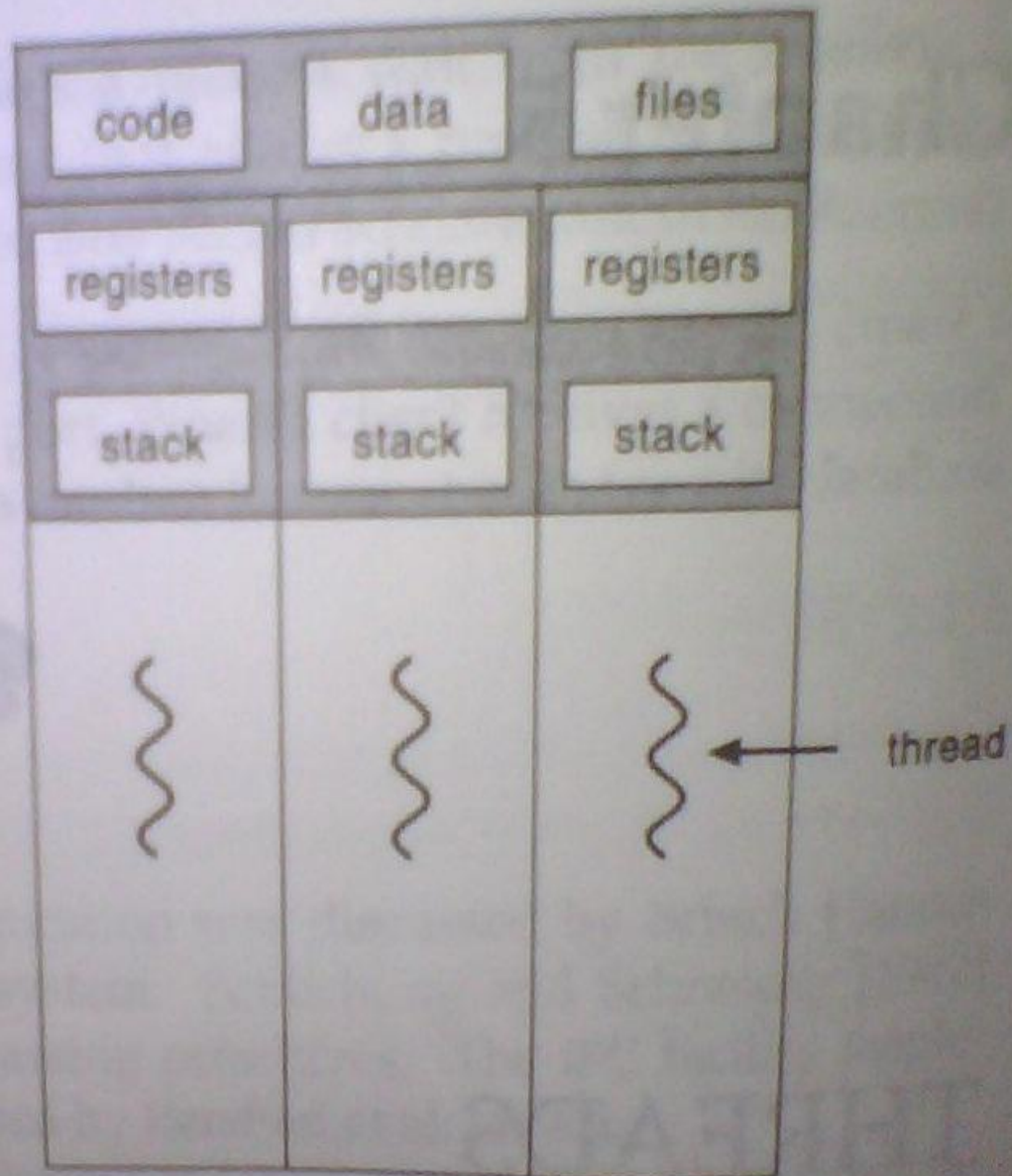
In the operating system UNIX, every process except process 0 (the swapper) is created when another process executes the `fork()` system call. The process that invoked `fork` is the parent process and the newly-created process is the child process. Every process (except process 0) has one parent process, but can have many child processes. The operating system kernel identifies each process by its process identifier. Process 0 is a special process that is created when the system boots; after forking a child process (process 1), process 0 becomes the swapper process (sometimes also known as the "idle task"). Process 1, known as *init*, is the ancestor of every other process in the system.

Threads

- A thread, also called a lightweight process (LWP), is a basic unit of CPU utilization; it consists of a thread ID, a program counter, a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.
- A traditional (or **heavyweight**) process has a single thread of control.
- If the process has multiple threads of control, it can do more than one task at a time. The term **multithreading** is used to describe the situation of allowing the multiple threads in same process.
- When multithreaded process is run on a single-CPU system, the threads take turns running as in the multiple processes.



single-threaded



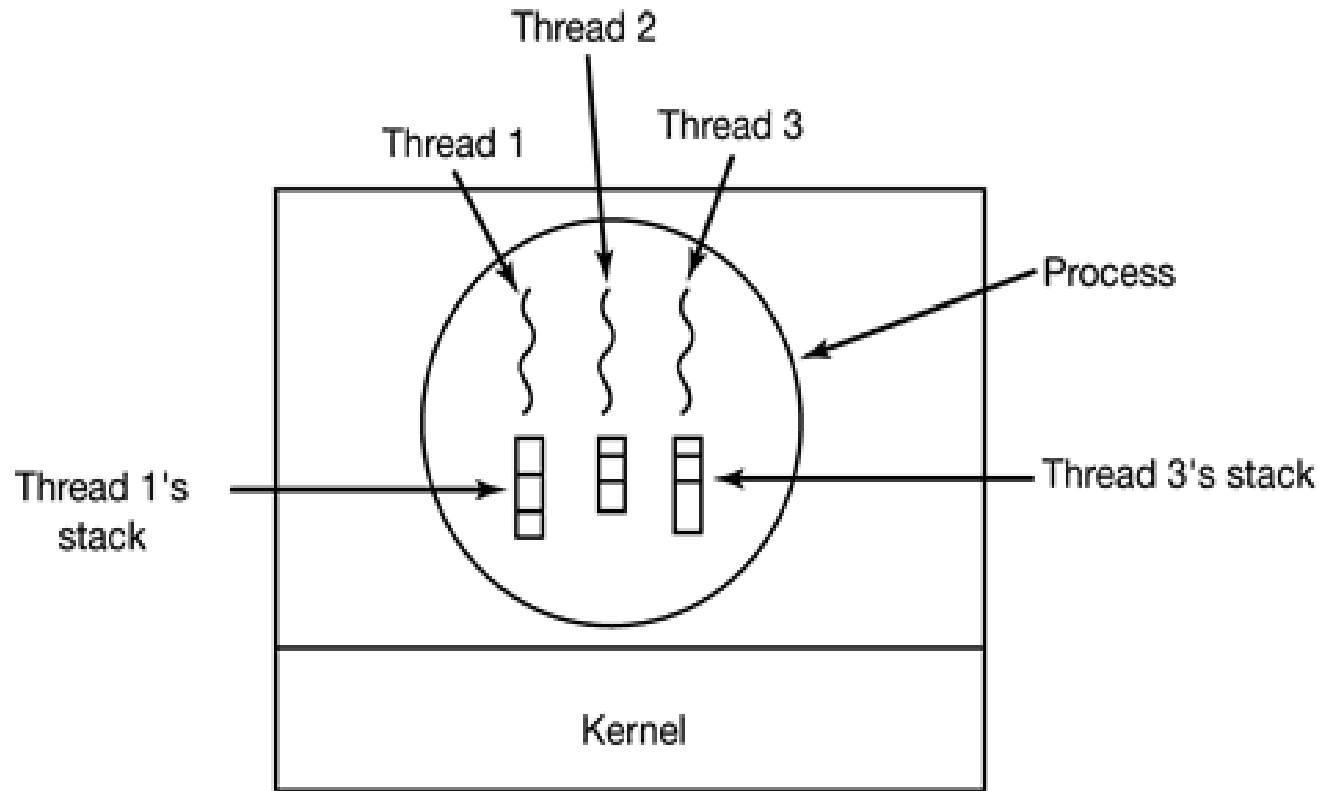
multithreaded

Figure Single- and multithreaded processes.

The Thread Model

- The *thread* has a program counter that keeps track of which instruction to execute next. It has registers, which hold its current working variables. It has a stack, which contains the execution history, with one frame for each procedure called but not yet returned from.
- Although a thread must execute in some process, the thread and its process are different concepts. Processes are used to group resources together; threads are the entities scheduled for execution on the CPU.

The Thread Model...



Example: Word Processor

Accept user input, display it on screen, spell check, auto-save, grammar check, etc.

The Thread Model...

- Like processes, a thread can be in any one of several states: **running**, **blocked**, **ready**, or **terminated**. A running thread currently has the CPU and is active. A blocked thread is waiting for some event to unblock it. A ready thread is scheduled to run and will as soon as its turn comes up. The transitions between thread states are the same as the transitions between process states.
- Advantages:
 - Responsiveness (Interactivity)
 - Resource sharing
 - Faster execution in multiprocessor CPUs

Thread Usage

- *Reading Assignment: Tanenbaum book (Pages: 95-100)
(How multithreading improves performance
of word processor and web server???)*

Thread Usage...

Case: An electronic spreadsheet

- *An electronic spreadsheet is a program that allows a user to maintain a matrix, some of whose elements are data provided by the user. Other elements are computed based on the input data using potentially complex formulas. When a user changes one element, many other elements may have to be recomputed. By having a background thread do the re-compute on, the interactive thread can allow the user to make additional changes while the computation is going on. Similarly, a third thread can handle periodic backups to disk on its own.*

Implementing Threads in User Space

There are two main ways to implement a threads package: in user space and in the kernel.

User Threads:

Thread management done by user-level threads library.

- Implemented as a library
- Library provides support for thread creation, scheduling and management with no support from the kernel. Because the kernel is unaware of user level threads, all thread creation and scheduling are done in user space without the need for kernel intervention. Thus user-level threads are fast to create and manage.
- When threads are managed in user space, each process needs its own private **thread table** to keep track of the threads in that process.
- If kernel is single threaded, blocking system calls will cause the entire process to block, even if other threads are available to run within the application. Example: POSIX **Pthreads** (*IEEE 1003.1c*), Mach **C-threads**, etc.

Implementing Threads in User Space...

Kernel Threads:

Supported by the Kernel

- Kernel performs thread creation, scheduling and management in kernel space. Because thread management is done by the kernel, they are slower to create and manage.
- Blocking system calls are no problem (The kernel can schedule another thread in the application for execution).
- Most contemporary OS's support these threads.
- Examples: WinX, Linux, etc.

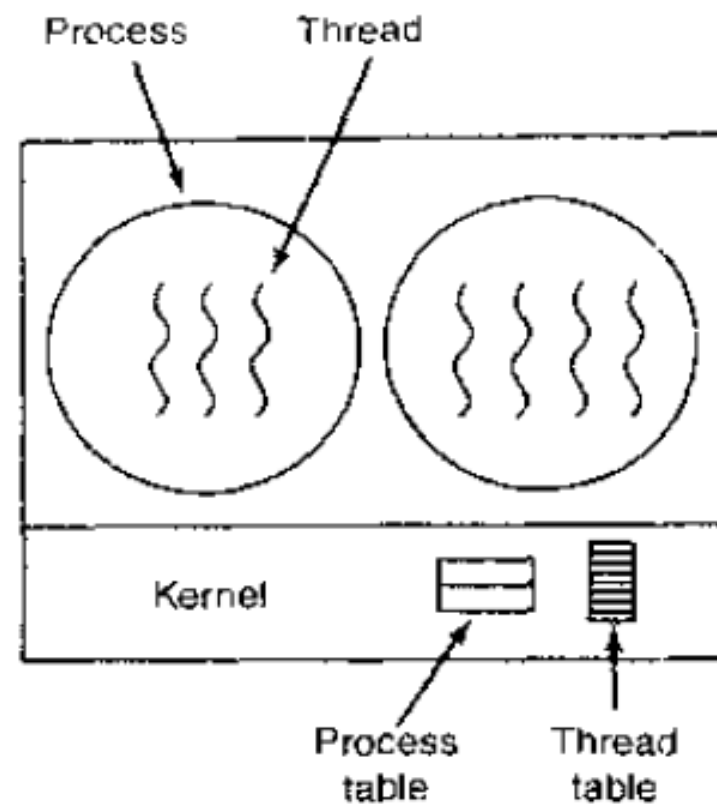
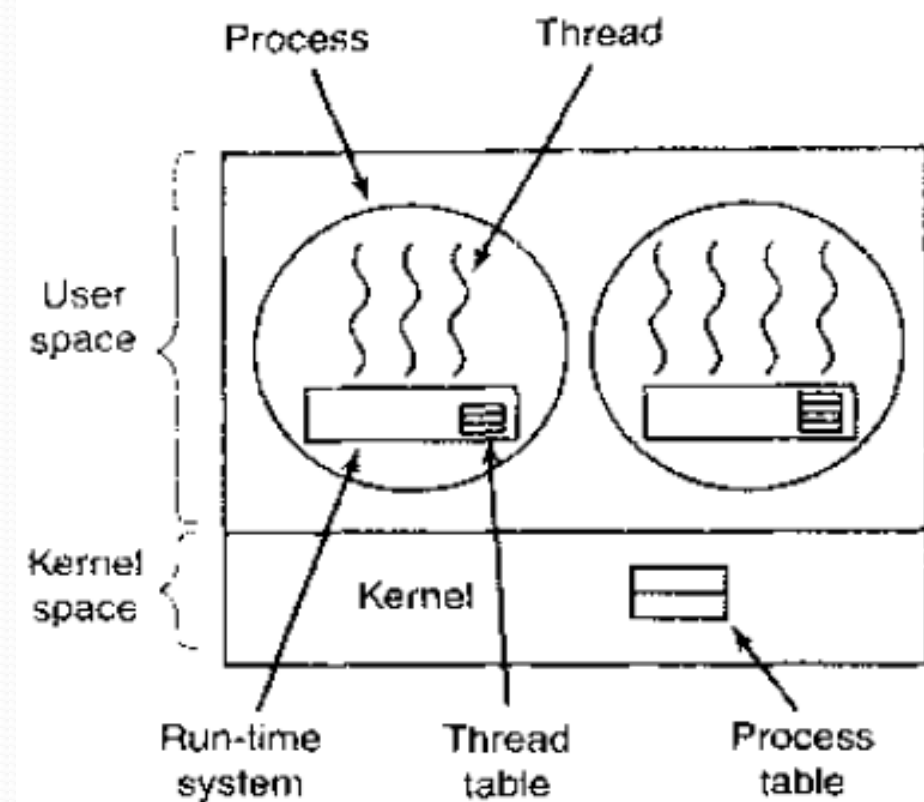


Figure: (a) A user-level threads package. (b) A threads package managed by the kernel.

TU Exam Question 2067

What is the problem with thread implementation in user space when any one of the threads gets blocked while performing I/O operation?

- ***Answer: A process may contain many threads. Threads can be implemented in user space and in kernel space. When implemented in user space, if any one of the threads gets blocked while performing an I/O operation, the kernel which is single threaded, not even knowing about the existence of threads, naturally blocks the entire process until the disk I/O is complete, even though other threads might be runnable at that time.***

TU Exam Question 2066

Why thread is necessary? In which circumstances user-level thread is better than Kernel level thread?

- **Hint:** [For applications that are essentially entirely CPU bound and rarely block, there is no point of having threads at all? Example: No one would like to compute the first n prime numbers or play chess using threads because there is nothing to be gained by doing it that way.]
- **Hint:** [For interactive systems that need regular thread switching without blocking system calls, user level threads are better]
- **Hint:** [Since kernel threads require some table space and stack space in the kernel, this can be a problem if there are a very large number of threads.]

Assignment #3

1. Describe how multithreading improves performance over a singled-threaded solution.
2. What are the two differences between the kernel level threads and user level threads? Which one has a better performance?
3. List the differences between processes and threads.
4. What resources are used when a thread is created? How do they differ from those used when a process is created?

Interprocess Communication (IPC)

- In inter-process communication, we look at the following three issues:
 - How one process can pass information to another.
 - How to make sure two or more processes do not get into each other's way when engaging in critical activities (suppose two processes each try to grab the last 1 MB of memory).
 - How to maintain the proper sequence when dependencies are present: if process *A* produces data and process *B* prints them, *B* has to wait until *A* has produced some data before starting to print.
- ***IPC provides the mechanism to allow the processes to communicate and to synchronize their actions.***

Race Condition

- Processes that are working together may share some common storage that each one can read and write.
- **Example:** a print spooler. When a process wants to print a file, it enters the file name in a special **spooler directory**. Another process, the **printer daemon**, periodically checks to see if there are any files to be printed, and if there are, it prints them and then removes their names from the directory.

Race Condition...

- Let the spooler directory has a number of slots, numbered 0, 1, 2, ..., each one capable of holding a file name. Also imagine that there are two shared variables, **out**, which points to the next file to be printed, and **in**, which points to the next free slot in the directory. These two variables may be kept on a two-word file available to all processes. At a certain instant, slots 0 to 3 are empty (the files have already been printed) and slots 4 to 6 are full (with the names of files queued for printing). Now let processes *A* and *B* decide they want to queue a file for printing i.e. two processes want to access shared memory at the same time.

Race Condition...

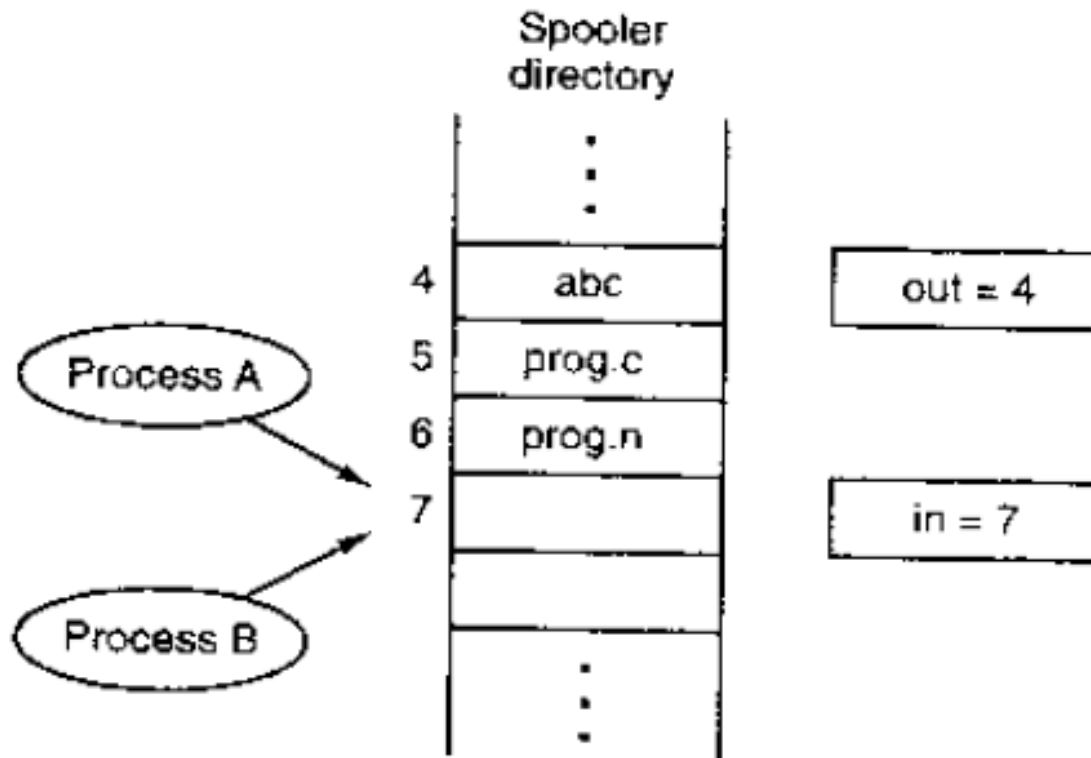


Figure : Two processes want to access shared memory at the same time.

Race Condition...

- ***Problem:*** Process *A* reads *in* and stores the value, 7, in a local variable called *next_free_slot*. Just then a clock interrupt occurs and the CPU switches to process *B*. Process *B* also reads *in*, and also gets a 7. It too stores it in *its* local variable *next_free_slot*. At this instant both processes think that the next available slot is 7. Process *B* now continues to run. It stores the name of its file in slot 7 and updates *in* = 8. Eventually, process *A* runs again, starting from the place it left off. It looks at *next_free_slot*, finds a 7 there, and writes its file name in slot 7, erasing the name that process *B* just put there. Then it computes *next_free_slot* + 1, which is 8, and sets *in* = 8. There is no problem with the *spooler directory*, so the *printer daemon* will not notice anything wrong, but process *B* will never receive any output.
- “Situations like this, where two or more processes are reading or writing some shared data and the final result depends on who runs precisely when, are called **race conditions**.”

Mutual Exclusion

- **Solution:** *The difficulty above occurred because process B started using one of the shared variables before process A was finished with it. Thus, prohibit more than one process from reading and writing the shared data at the same time - **Mutual Exclusion**.*
- **Mutual Exclusion:** *Some way of making sure that if one process is using a shared variable or files, the other processes will be excluded from doing the same thing.*

Critical Region

- *We need to avoid race conditions.*
- *The part of time a process is busy doing internal computations and other things, it does not lead to race condition. But, sometimes a process have to access shared memory or files that can lead to races. That part of the program where the shared memory is accessed is called the **critical region** or **critical section**.*
- *So, code executed by the process can be grouped into sections, some of which require access to shared resources, and others that do not. The section of the code that require access to shared resources is called **critical section/region**.*

Critical Region...

```
while(true)  
  {  
    entry_section  
      critical_section  
    exit_section  
      remainder_section  
  }
```

Fig: General structure of a typical process P_i

- When a process is accessing a shared modifiable data, the process is said to be in critical section.
- All other processes (those access the same data) are excluded from their own critical region.
- All other processes may continue executing outside their CR.
- When a process leaves its critical section, then another process waiting to enter its own CR should be allowed to proceed.

Critical Region...

The following conditions must be satisfied:

1. No two processes may be simultaneously inside their CRs (*mutual exclusion*).
2. No assumptions may be made about the speeds or number of CPUs.
3. No process running outside its CR may block other processes.
4. No process should have to wait forever to enter its CR.

Critical Region...

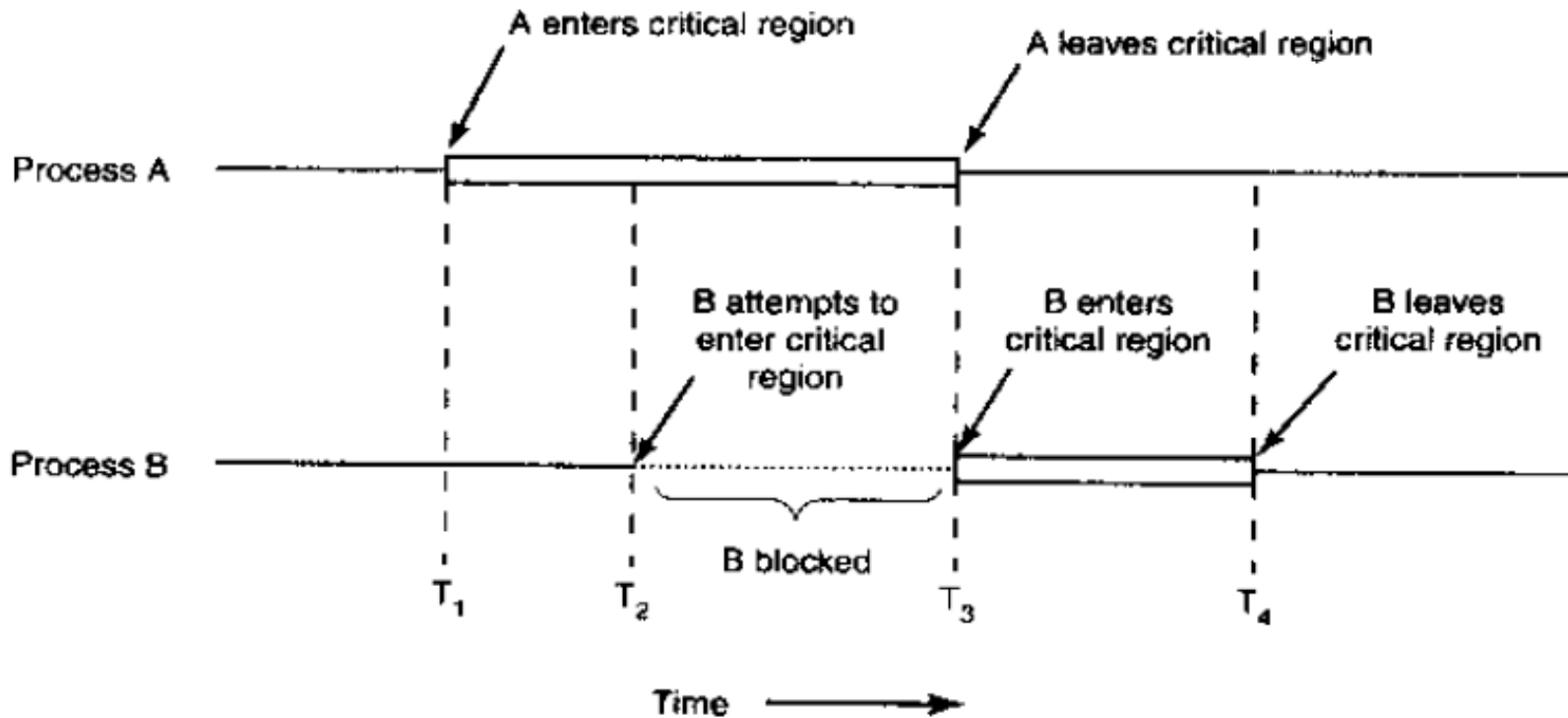


Figure: Mutual exclusion using critical regions.

TU Exam Question 2068

- What is critical section problem? Why executing critical selection must be mutually exclusive? Explain.

Mutual Exclusion with Busy Waiting

There are various schemes for achieving *mutual exclusion*, so that while one process is busy updating shared memory in its critical region, no other process will enter its critical region and cause trouble.

- **Disabling Interrupts**

In this scheme, each process disables all interrupts just after entering its critical region and re-enables them just before leaving it. With interrupts disabled, no clock interrupts can occur. Since, the CPU is only switched from process to process as a result of interrupts, and with interrupts turned off the CPU will not be switched to another process. Thus, once a process has disabled interrupts, it can examine and update the shared memory without fear that any other process will interfere.

Mutual Exclusion with Busy Waiting

Disabling Interrupts...

Problem: *This scheme is unattractive because it is unwise to give user processes the power to turn off interrupts. Suppose that one of them did it and never turned them on again? That could be the end of the system. Also, if the system is a multiprocessor, with two or more CPUs, disabling interrupts affects only the CPU that executed the disable instruction. The other CPUs will continue running and can access the shared memory.*

Mutual Exclusion with Busy Waiting

- Lock Variables

In this scheme, a single, shared (lock) variable, is used which is initially 0. When a process wants to enter its CR, it first tests the lock. If the lock is 0, the process sets it to 1 and enters the critical region. If the lock is already 1, the process just waits until it becomes 0. Thus, a 0 means that no process is in its critical region, and a 1 means that some process is in its critical region.

Problem: *Same as spooler directory problem. Suppose that one process reads the lock and sees that it is 0. Before it can set the lock to 1, another process is scheduled, runs, and sets the lock to 1. When the first process runs again, it will also set the lock to 1, and two processes will be in their critical regions at the same time (mutual exclusion violated).*

Mutual Exclusion with Busy Waiting

- *Strict Alternation*

*In this scheme, processes share a common integer variable **turn**. If **turn** == **i**, then process P_i is allowed to execute in its critical region, and if **turn** == **j**, then process P_j is allowed to execute.*

```
While (true){  
    while(turn!=i); /*loop */  
    critical_section();  
    turn = j;  
    noncritical_section();  
}
```

Process P_i

```
While (true){  
    while(turn!=j); /*loop*/  
    critical_section();  
    turn = i;  
    noncritical_section();  
}
```

Process P_j

Mutual Exclusion with Busy Waiting

Strict Alternation...

- **Problem:** When process i leaves the critical region, it sets $turn$ to j , to allow process j to enter its critical region. Suppose that process j finishes its critical region quickly, so both processes are in their noncritical regions, with $turn$ set to i . Now process i executes its whole loop quickly, exiting its critical region and setting $turn$ to j . At this point $turn$ is j and both processes are executing in their noncritical regions.
- Suddenly, process i finishes its noncritical region and goes back to the top of its loop. Unfortunately, it is not permitted to enter its critical region now, because $turn$ is j and process j is busy with its noncritical region. It hangs in its while loop until process j sets $turn$ to i ----- **Violation of condition 3**

No process running outside its CR may block other processes.

Mutual Exclusion with Busy Waiting

- *Peterson's Algorithm*

```
#define FALSE 0
#define TRUE 1
#define N      2                /* number of processes */

int turn;                       /* whose turn is it? */
int interested[N];              /* all values initially 0 (FALSE) */

void enter_region(int process); /* process is 0 or 1 */
{
    int other;                  /* number of the other process */

    other = 1 - process;        /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;             /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process) /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}
```

Fig: Peterson's solution for achieving mutual exclusion

Mutual Exclusion with Busy Waiting

Peterson's Algorithm...

- **Explanation:** *In this scheme, before using shared variables (i.e., before entering its CR), each process calls **enter_region** with its own process number, 0 or 1, as parameter. After it is finished with the CR, the process calls **leave_region** to indicate that it is done and to allow the other process to enter its CR, if needed.*

*Initially neither process is in its CR. Now process 0 calls **enter_region**. It indicates its interest by setting its array element and sets turn to 0. Since process 1 is not interested, **enter_region** returns immediately. If process 1 now calls **enter_region**, it will hang there until `interested[0]` goes to `FALSE`, an event that only happens when process 0 calls **leave_region** to exit the critical region.*

Problem: *Difficult to program for n-processes system and less efficient.*

Mutual Exclusion with Busy Waiting

- Hardware Solution – The TSL Instruction...

When lock is 0, any process may set it to 1 using the TSL instruction and then read or write the shared memory. When it is done, the process sets lock back to 0 using an ordinary move instruction.

enter_region:

TSL REGISTER,LOCK	copy lock to register and set lock to 1
CMP REGISTER,#0	was lock zero?
JNE enter_region	if it was non zero, lock was set, so loop
RET	return to caller; critical region entered

leave_region:

MOVE LOCK,#0	store a 0 in lock
RET	return to caller

Figure: Entering and leaving a critical region using the TSL instruction.

Note: Before entering its critical region, a process calls enter_region and after the critical region work is finished, the process calls leave_region.

Problem: Difficulty in hardware design

Busy Waiting Alternate???

- Both Peterson's solution and the solution using TSL are correct, but both have the defect of requiring busy waiting.
- **Busy Waiting:** When a process wants to enter its critical region, it checks to see if the entry is allowed. If it is not, the process just sits in a tight loop waiting until it is allowed.
- **Problem:** Waste of CPU time.
- **Alternate: Blocking** instead of wasting CPU time when they are not allowed to enter their critical regions ----- ***sleep and wakeup.***
- *Sleep is a system call that causes the caller to block, i.e., be suspended until another process wakes it up. The wakeup call has one parameter, the process to be awakened.*

TU Model Question

- Explain the difference between Busy Waiting and Blocking.

Sleep and Wakeup

- **The Producer-Consumer Problem (Bounded-Buffer Problem)**

To show how IPC works in this case, let us consider that two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out.

If producer inserts item rapidly, the buffer will go full and it will go to the sleep, to be awakened when the consumer has removed one or more items. Similarly, if the consumer wants to remove an item from the buffer and sees that the buffer is empty, it goes to sleep until the producer puts something in the buffer and wakes it up.

Sleep and Wakeup...

The Producer-Consumer Problem (Bounded-Buffer Problem)...

To keep track of the number of items in the buffer, we need a variable, **count**. If the maximum number of items the buffer can hold is N , the producer's code will first test to see if **count** is N . If it is, the producer will go to sleep; if it is not, the producer will add an item and increment **count**.

The consumer first tests **count** to see if it is 0. If it is, go to sleep, if it is nonzero, remove an item and decrement the **count**. Each of the processes also tests to see if the other should be awakened, and if so, wakes it up. The code for both producer and consumer is shown below:

Sleep and Wakeup

Producer-Consumer Problem

```
#define N = 100    /* number of slots in the buffer*/
int count = 0;     /* number of item in the buffer*/
void producer(void)
{
    int item;
    while(TRUE){    /* repeat forever*/
        item = produce_item(); /* generate next item*/
        if (count == N) sleep(); /*if buffer is
                                   full go to sleep*/
        insert_item(item); /* put item in buffer */
        count = count + 1; /*increment count */
        if (count == 1) wakeup(consumer);
                                   /* was buffer empty*/
    }
}
```

```
void consumer(void)
{
    int item;
    while(TRUE){    /* repeat forever*/
        if(count == 0)sleep(); /* if buffer is
                                   empty go to sleep*/
        item = remove_item(); /*take item out
                                   of buffer*/
        count = count - 1; /*decrement count*/
        if (count == N-1)wakeup(producer);
                                   /*was buffer full ?*/
        consume_item();    /*print item*/
    }
}
```

Sleep and Wakeup

The Producer-Consumer Problem (Bounded-Buffer Problem)...

Problem:

-leads to race condition as in spooler directory.

Example: When the buffer is empty, the consumer just reads count and let the quantum gets expired. The producer inserts an item in the buffer, increments count and wakes up the consumer. The consumer not yet asleep, so the wakeup signal is lost. The consumer has the count value 0 from the last read, so it goes to sleep. Producer keeps on producing, fills the buffer and goes to sleep. Both will sleep forever **DEADLOCK**

Solution: Save the wakeup signal that was lost.

Semaphores

- E. W. Dijkstra (1965) suggested using an integer variable to count the number of wakeups, called a ***semaphore***. It could have the value 0, indicating no wakeups were saved, or some positive value if one or more wakeups were pending.
- Dijkstra proposed having two operations, **down** and **up** (originally he proposed **P** and **V** in Dutch and sometimes known as **wait** and **signal**).
- **Down**: The down operation on a semaphore checks to see if the value is greater than 0.
 - Yes - decrement the value (i.e. uses one stored wakeup) and continues.
 - No - process is put to sleep without completing down.
 - (Checking value, changing it, and possibly going to sleep, is all done as a single atomic action)
- **Up**: *increments the value; if one or more processes were sleeping, unable to complete earlier down operation, one of them is chosen and is allowed to complete its down.*

Semaphores...

```
typedef int semaphore S;  
semaphore mutex = 1;  
void down(S)  
{  
    if(S > 0)  
        S--;  
    else  
        sleep();  
}  
void up(S)  
{  
    if(one or more processes are sleeping on S)  
        one of these process is proceed;  
    else S++;  
}
```

```
while(TRUE)  
{  
    down(mutex);  
    critical_region();  
    up(mutex);  
    noncritical_region();  
}
```

Producer-Consumer using semaphore

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

/* number of slots in the buffer */
/* semaphores are a special kind of int */
/* controls access to critical region */
/* counts empty buffer slots */
/* counts full buffer slots */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        down(&empty);
        down(&mutex);
        insert_item(item);
        up(&mutex);
        up(&full);
    }
}

/* TRUE is the constant 1 */
/* generate something to put in buffer */
/* decrement empty count */
/* enter critical region */
/* put new item in buffer */
/* leave critical region */
/* increment count of full slots */

void consumer(void)
{
    int item;

    while (TRUE) {
        down(&full);
        down(&mutex);
        item = remove_item();
        up(&mutex);
        up(&empty);
        consume_item(item);
    }
}
```

Figure : The producer-consumer problem using semaphores.

Producer-Consumer using semaphore

- This solution uses three semaphores: one called **full** for counting the number of slots that are full, one called **empty** for counting the number of slots that are empty, and one called **mutex** to make sure the producer and consumer do not access the buffer at the same time. The **full** is initially 0, **empty** is initially equal to the number of slots in the buffer, and **mutex** is initially 1. (*Semaphores that are initialized to 1 and used by two or more processes to ensure that only one of them can enter its critical region at the same time are called **binary semaphores***). Since each process does a down just before entering its critical region and an up just after leaving it, mutual exclusion is guaranteed. The other semaphores are for **synchronization**. The **full** and **empty** semaphores ensure that the producer stops running when the buffer is full, and the consumer stops running when it is empty.

Use of semaphore

1. *To deal with n-process critical-section problem:*

The n processes share a semaphore, (e.g. mutex) initialized to 1.

2. *To solve the various synchronizations problems:*

For example, let there be two concurrently running processes: P₁ with statement S₁ and P₂ with statement S₂. Suppose it is required that S₂ must be executed after S₁ has completed. This problem can be implemented by using a common semaphore, synch, initialized to 0.

P₁: S₁;

up(synch);

P₂: down(synch);

S₂;

Criticality using semaphores

- All processes share a common semaphore variable ***mutex***, initialized to 1. Each process must execute ***down(mutex)*** before entering CR, and ***up(mutex)*** afterward. When this sequence is not observed-----
- **Problem 1:** When a process interchange the order of ***down*** and ***up*** operation on ***mutex***, causes multiple processes in CR simultaneously => **Violation of mutual exclusion.**

up(mutex);
critical_section();
down(mutex);

Problem 2: Let the two downs in the producer's code were reversed in order, so ***mutex*** was decremented before ***empty*** instead of after it. If the buffer were completely full (i.e. ***empty*** = 0), the producer would block, with ***mutex*** = 0. Now, when the consumer tries to access the buffer, it would do a down on ***mutex***, now 0, and block too. Both processes would stay blocked forever => **DEADLOCK.**

Lesson learned: A subtle error is capable to bring whole system grinding halt!!! Thus using semaphore is very critical for the programmer.

Solution: MONITOR

TU Exam Question 2068

- “Using semaphore is very critical for programmer”. Do you support this statement? If yes, prove the statement with some fact. If not, put your view with some logical facts against the statement.

Monitor

- **Monitor** is a higher-level inter-process synchronization primitive.
- A monitor is a programming language construct that guarantees appropriate access to the CR.
- It is a collection of procedures, variables, and data structures that are all grouped together in a special kind of module or package.
- Processes that wish to access the shared data, do through the execution of monitor functions.
- Only one process can be active in a monitor at any instant.
- Compiler level management.

Monitor...

Skeleton of a monitor:

```
monitor monitor_name  
{  
    shared variable declarations;  
    procedure p1(...){ ..... }  
    procedure p2(...){ .... }  
    ... ..  
    procedure pn(...){ ...}  
    {initialization code;}  
}
```

```

monitor ProducerConsumer
{
int count;
condition full, empty;
void insert(int item)
{
    if(count==N) wait(full);
    insert_item(item);
    count++;
    if(count==1) signal(empty);
}
void remove()
{
    if(count==0) wait(empty);
    remove_item();
    count--;
    if(count==N-1) signal(full);
}
    count=0;
};

```

```

void producer()
{
    while(TRUE)
    {
        item=produce_item();
        ProducerConsumer.insert(item);
    }
}

void consumer()
{
    while(TRUE)
    {
        item=ProducerConsumer.remove();
        consume_item();
    }
}

```

Fig: An outline of the producer-consumer problem with monitors.

Monitor...

The operations **wait** and **signal** are similar to **sleep** and **wakeup**, but with one crucial difference: **sleep** and **wakeup** failed because while one process was trying to go to sleep, the other one was trying to wake it up. With monitors, that cannot happen. The automatic mutual exclusion on monitor procedures guarantees that if, say, the producer inside a monitor procedure discovers that the buffer is full, it will be able to complete the **wait** operation without having to worry about the possibility that the scheduler may switch to the consumer just before the **wait** completes. The consumer will not even be let into the monitor at all until the **wait** is finished.

Problem with monitors

1. Lack of implementation in most commonly used programming languages like C.
2. Both semaphore and monitors are used to hold mutual exclusion in multiple CPUs that all have a common memory. But in distributed system consisting of multiple CPUs with its own private memory, connected by LAN, none of these primitives are applicable (i.e. none of the primitives allow information exchange between machines).

Message Passing

- In this scheme, IPC is based on two primitives: send and receive.

`send(destination, &message);`

`receive(source, &message);`

- The send and receive calls are normally implemented as operating system calls accessible from many programming language environments.

Reading Assignment

- *Producer-Consumer with Message Passing (Pages 142-143) --- Textbook (Tanenbaum)*

Producer-Consumer with Message Passing

```
#define N 100                                /*number of slots in the buffer*/
void producer(void)
{   int item;
    message m;                                /*message buffer*/
    while (TRUE){
        item = produce_item();    /*generate something */
        receive(consumer, &m);    /*wait for an empty to arrive*/
        build_message(&m, item); /*construct a message to send*/
        send(consumer, &m); }
}
void consumer(void)
{   int item;
    message m;
    for(i = 0; i<N; i++) send(producer, &m);    /*send N empties*/
    while(TRUE){
        receive(producer, &m);    /* get message containing item*/
        item = extract_item(&m);    /* extract item from message*/
        send(producer, &m);    /* send back empty reply*/
        consume_item(item);    /*do something with item*/
    }
}
```

Producer Consumer with Message Passing...

- No shared memory.
- Messages sent but not yet received are buffered automatically by OS, it can save N messages.
- The total number of messages in the system remains constant, so they can be stored in given amount of memory known in advance.
- If the producer works faster than the consumer, all the messages will end up full, waiting for the consumer: the producer will be blocked, waiting for an empty to come back. If the consumer works faster, all the messages will be empties waiting for the producer to fill them up: the consumer will be blocked, waiting for a full message.

TU Model Question

- Give briefly at least three different ways of establishing Interprocess communication?

Classical IPC Problems

- The Dining Philosophers Problem
- The Readers-Writers Problem
- The Sleeping-Barber Problem
- The Cigarette-Smokers Problem

The Dining Philosophers Problem

- The *dining philosophers problem* is a synchronization problem posed and solved by Dijkstra in 1965.
- **Scenario:** Five philosophers are seated around a circular table for their lunch. Each philosopher has a plate of spaghetti. The spaghetti is so slippery that a philosopher needs two forks to eat it. Between each pair of plates is one fork. The philosophers alternate between thinking and eating.

When a philosopher gets hungry, she tries to acquire her left and right fork, one at a time, in either order. If successful in acquiring two forks, she eats for a while, then puts down the forks, and continues to think.

- **Problem:** What is the solution (program) for each philosopher that does what it is supposed to do and never gets stuck?

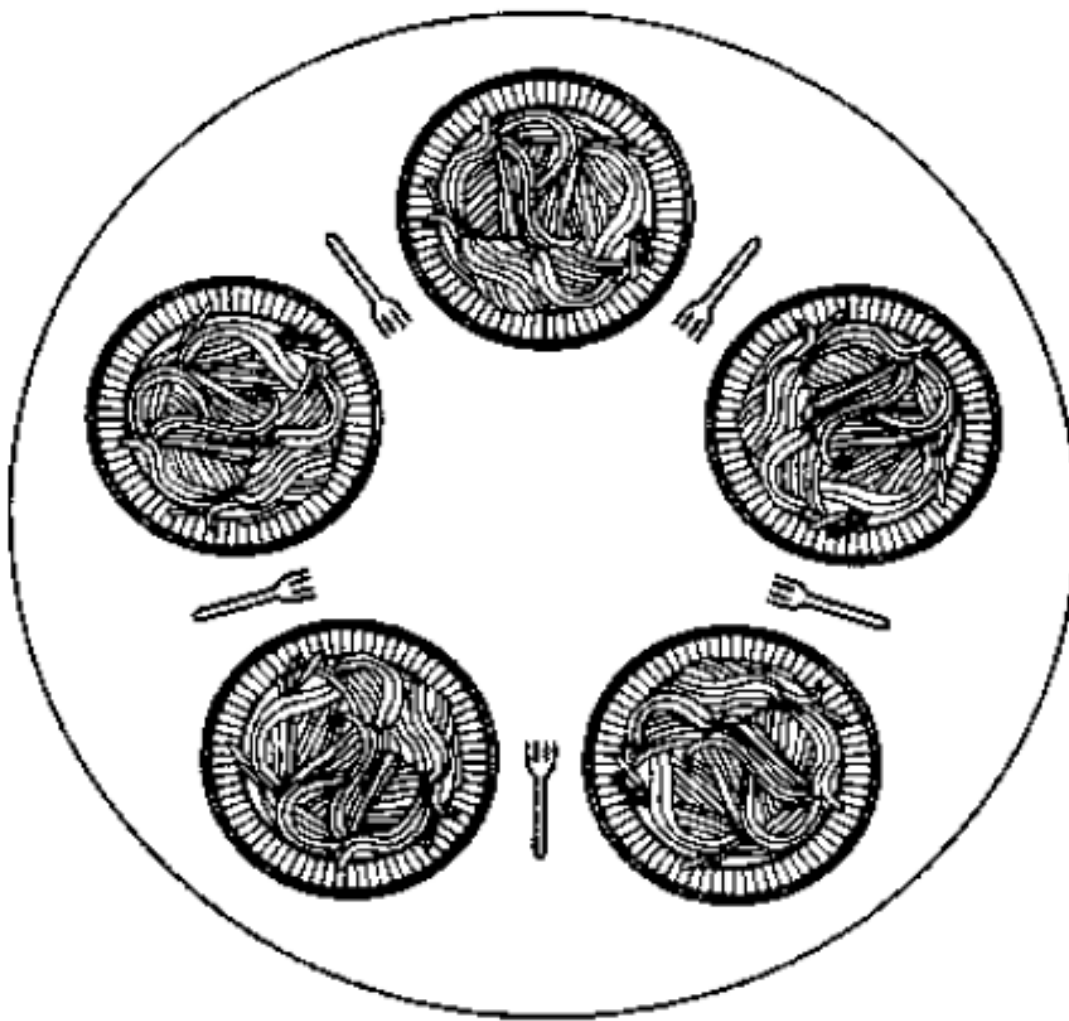


Figure: Lunch time in the Philosophy Department.

Solution to Dining Philosophers Problem

- **Attempt 1:** When a philosopher is hungry, she picks up her left fork and waits for right fork. When she gets it, she eats for a while and then puts both forks back to the table.

```
#define N 5                                     /* number of philosophers */
void philosopher(int i)                         /* i: philosopher number, from 0 to 4 */
{
    while (TRUE)
    {
        think( );                             /* philosopher is thinking */
        take_fork(i);                          /* take left fork */
        take_fork((i+1) % N);                  /* take right fork; % is modulo operator */
        eat();                                 /* yum-yum, spaghetti */
        put_fork(i);                           /* put left fork back on the table */
        put_fork((i+1) % N);                   /* put right fork back on the table */
    }
}
```

- **Problem:** What will happen, if all five philosophers take their left forks simultaneously???.....**DEADLOCK**

Solution to Dining Philosophers Problem

- **Attempt 2:** After taking the left fork, checks for right fork. If it is not available, the philosopher puts down the left one, waits for some time, and then repeats the whole process.
- **Problem:** What will happen, if all five philosophers take their left fork simultaneously???... **STARVATION**

*“A situation in which all the programs continue to run indefinitely but fail to make any progress is called **starvation**.”*

Solution to Dining Philosophers Problem

- **Attempt 3:** Protect the five statements following the call to *think* by a **binary semaphore**. Before starting to acquire forks, a philosopher would do a down on *mutex*. After replacing the forks, she would do an up on *mutex*.
- **Problem:** Adequate solution but not perfect solution since only one philosopher can be eating at any instant. With five forks available, we should be able to allow two philosophers to eat at the same time.

Solution to Dining Philosophers Problem

- **Attempt 4:** Using **semaphore** for each philosopher, a philosopher moves only in eating state if neither neighbor is eating ***Perfect Solution.***

Code given on next slide:

```

#define N          5
#define LEFT      (i+N-1)%N
#define RIGHT     (i+1)%N
#define THINKING  0
#define HUNGRY    1
#define EATING    2
typedef int semaphore;
int state[N];
semaphore mutex = 1;
semaphore s[N];

void philosopher(int i)
{
    while (TRUE) {
        think();
        take_forks(i);
        eat();
        put_forks(i);
    }
}

void take_forks(int i)
{
    down(&mutex);
    state[i] = HUNGRY;
    test(i);
    up(&mutex);
    down(&s[i]);
}

void put_forks(i)
{
    down(&mutex);
    state[i] = THINKING;
    test(LEFT);
    test(RIGHT);
    up(&mutex);
}

void test(i)
{
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
        state[i] = EATING;
        up(&s[i]);
    }
}

/* number of philosophers */
/* number of i's left neighbor */
/* number of i's right neighbor */
/* philosopher is thinking */
/* philosopher is trying to get forks */
/* philosopher is eating */
/* semaphores are a special kind of int */
/* array to keep track of everyone's state */
/* mutual exclusion for critical regions */
/* one semaphore per philosopher */

/* i: philosopher number, from 0 to N-1 */

/* repeat forever */
/* philosopher is thinking */
/* acquire two forks or block */
/* yum-yum, spaghetti */
/* put both forks back on table */

/* i: philosopher number, from 0 to N-1 */

/* enter critical region */
/* record fact that philosopher i is hungry */
/* try to acquire 2 forks */
/* exit critical region */
/* block if forks were not acquired */

/* i: philosopher number, from 0 to N-1 */

/* enter critical region */
/* philosopher has finished eating */
/* see if left neighbor can now eat */
/* see if right neighbor can now eat */
/* exit critical region */

/* i: philosopher number, from 0 to N-1 */

```

Figure : A solution to the dining philosophers problem.

Explanation:

- This solution is deadlock-free and allows the maximum parallelism for an arbitrary number of philosophers. It uses an array, *state*, to keep track of whether a philosopher is *eating*, *thinking*, or *hungry* (trying to acquire forks). A philosopher may move only into *eating* state if neither neighbor is *eating*. Philosopher *i*'s neighbors are defined by the macros *LEFT* and *RIGHT*. In other words, if *i* is 2, *LEFT* is 1 and *RIGHT* is 3.
- The program uses an array of semaphores, one per philosopher, so hungry philosophers can block if the needed forks are busy. Note that each process runs the procedure *philosopher* as its main code, but the other procedures, *take_forks*, *put_forks*, and *test* are ordinary procedures and not separate processes.

TU Exam Question 2066

- Define the term semaphore. How does semaphore help in dining philosophers problem? Explain.

Process Scheduling

By switching the processor among the processes, the OS can make the computer more productive. In a multiprogramming environment, multiple processes are in memory and competing for the CPU at the same time. If only one CPU is available, a choice has to be made which process to run next. The part of the operating system that makes the choice is called the **scheduler** and the algorithm it uses is called the **scheduling algorithm**.

(Q) What to schedule?

(A) Which process is given control of the CPU and for how long?

Process Behavior

- Process execution consists of a cycle of CPU execution and I/O wait. It begins with a CPU burst that is followed by I/O burst, then another CPU burst, then another I/O burst... ..
- CPU-bound processes: Processes that use CPU until the quantum expire.
- I/O-bound processes: Processes that use CPU briefly and generate I/O request.
- CPU-bound processes have a long CPU-burst and rare I/O waits while I/O-bound processes have short CPU burst and frequent I/O waits.
- Key idea: when I/O bound process wants to run, it should get a chance quickly.

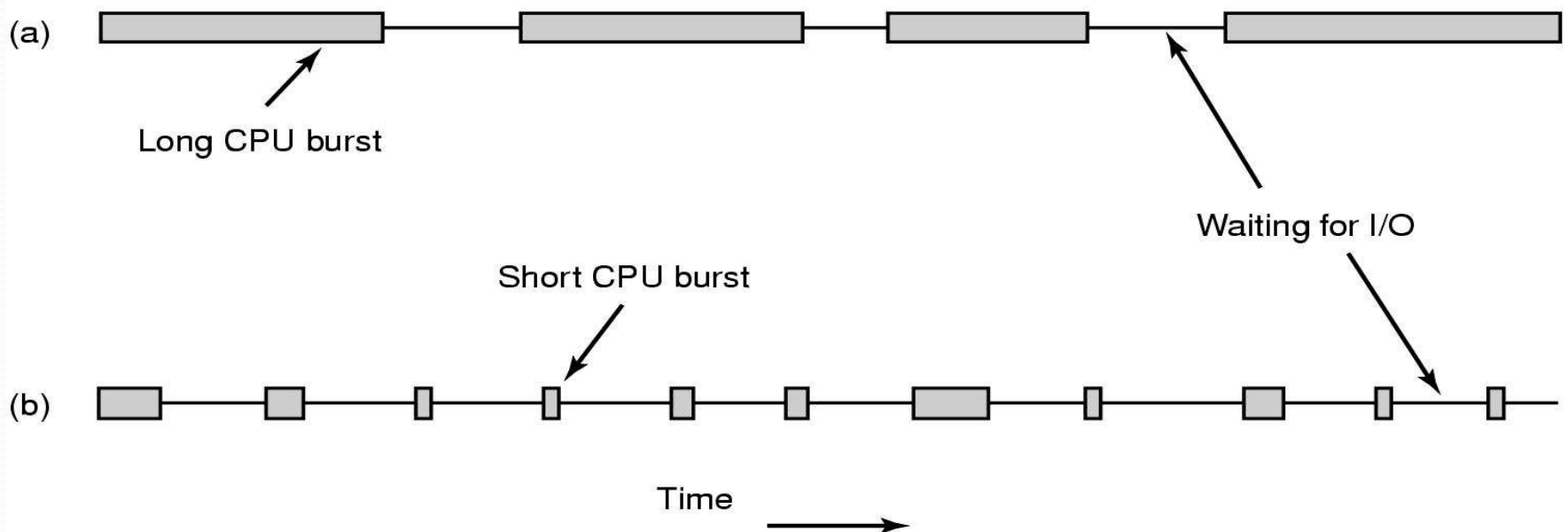


Figure: Bursts of CPU usage alternate with periods of waiting for I/O.
(a) A CPU-bound process. (b) An I/O-bound process.

TU Exam Question 2066

- How can you define the term process scheduling? Differentiate between I/O bound process and CPU bound process.

When to Schedule?

1. When a new process is created, a decision needs to be made whether to run the parent process or the child process. Since both processes are in ready state, it is a normal scheduling decision and it can go either way.
2. When a process terminates, some other process must be chosen from the set of ready processes.
3. When a process blocks on I/O, on a semaphore, or for some other reason, another process has to be selected to run.
4. When an I/O interrupt occurs, a scheduling decision needs to be made. If the interrupt came from an I/O device that has now completed its work, some process that was blocked waiting for the I/O may now be ready to run. It is up to the scheduler to decide if the newly ready process should be run, if the process that was running at the time of the interrupt should continue running, or if some third process should run.
5. When quantum expires.

Types of scheduling algorithms

Scheduling algorithms can be divided into two categories with respect to how they deal with clock interrupts.

- Preemptive scheduling algorithm
- Non-preemptive scheduling algorithm

Preemptive vs Non-preemptive Scheduling

Nonpreemptive:

- A **nonpreemptive** scheduling algorithm picks a process to run and then just lets it run until it blocks (either on I/O or waiting for another process) or until it voluntarily releases the CPU. Even if it runs for hours, it will not be forcefully suspended. In effect, no scheduling decisions are made during clock interrupts. After clock interrupt processing has been completed, the process that was running before the interrupt is always resumed.
- Treatment of all processes is fair.
- Response times are more predictable.
- Useful in real-time system.
- Shorter jobs are made to wait by longer jobs - no priority

Preemptive:

- A **preemptive** scheduling algorithm picks a process and lets it run for a maximum of some fixed time. If it is still running at the end of the time interval, it is suspended and the scheduler picks another process to run (if one is available). Doing preemptive scheduling requires having a clock interrupt occur at the end of the time interval to give control of the CPU back to the scheduler.
- Useful in systems in which high-priority processes require rapid attention.
- In timesharing systems, preemptive scheduling is important in guaranteeing acceptable response times.
- High overhead.

Scheduling Criteria

- The scheduler has to identify the process whose selection will result in the best possible (**optimal**) system performance.
- There are various criteria for comparing scheduling algorithms:
 - CPU Utilization: Try to keep the CPU busy all the time.
 - Throughput: Number of jobs per unit time that the system completes.
 - Turnaround Time: Time from the moment that a job is submitted until the moment it is completed.
 - Waiting Time: Total time needed to get CPU.
 - Response Time: Time from the submission of a request and getting the first response. Requests should be responded quickly.
 - Fairness: Giving each processes a fair share of the CPU.
 - Priorities: High priority jobs should get CPU first.

Scheduling Algorithms

- *First-Come First-Served*
- *Shortest Job First*
- *Shortest Remaining Time Next*
- *Round-Robin Scheduling*
- *Priority Scheduling*
- *Multiple Queues*

First-Come First-Served (FCFS)

- Processes are scheduled in the order they are received.
- Once a process gets the CPU, it runs to completion – Nonpreemptive.
- Can be easily implemented by managing a simple queue or by storing the time the process was received.
- Fair to all processes.

Problems:

- No guarantee of good response time.
- Large average waiting time.
- Not applicable for interactive system.



Shortest Job First (SJF)

- The processing times are known in advance.
- SJF selects the process with shortest expected processing time. In case of tie, FCFS scheduling is used.
- The decision policies are based on the CPU burst time.

Advantages:

- Reduces the average waiting time over FCFS.
- Favors short jobs at the cost of long jobs.

Problems:

- Estimation of run time to completion... Accuracy?
- Not applicable in timesharing system.

An Example

Scenario: Consider the following set of processes that arrive at time 0, with length of CPU-burst time in milliseconds.

Processes	Burst time
P1	24
P2	3
P3	3

if the processes arrive in the order P1, P2, P3 and are served in FCFS order,

P1	P2	P3
----	----	----

The average waiting time is $(0 + 24 + 27)/3 = 17$.

if the processes are served in SJF

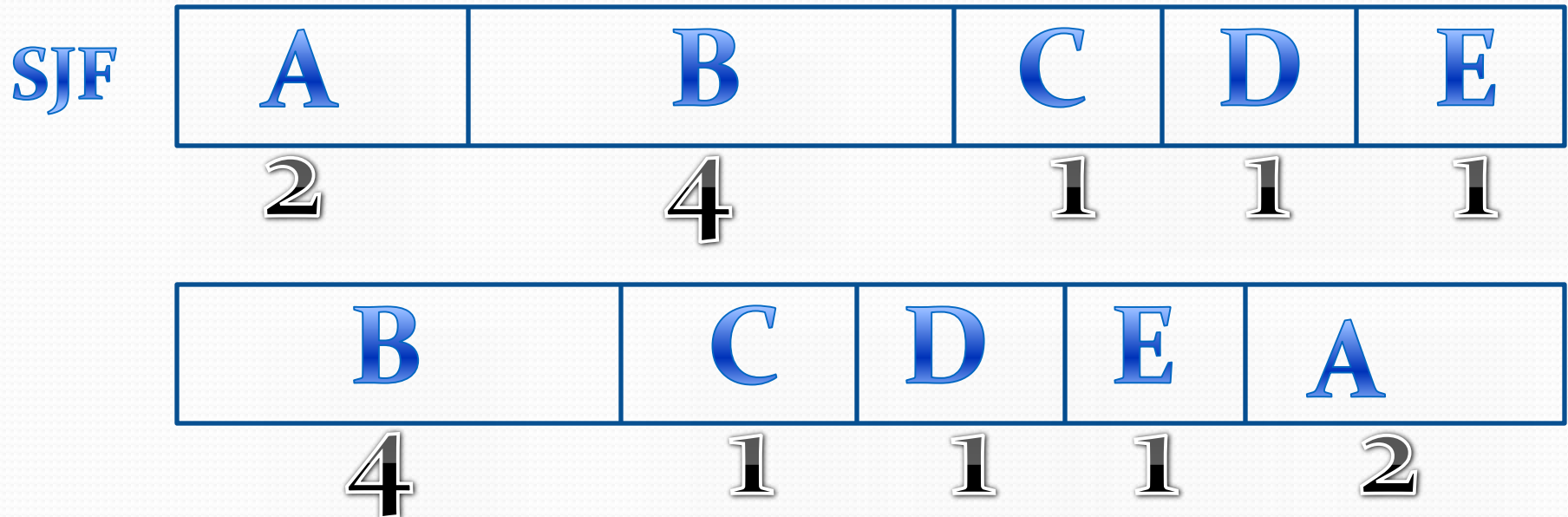
P2	P3	P1
----	----	----

The average waiting time is $(6 + 0 + 3)/3 = 3$.

- In FCFS, the turnaround time for $P_1 = 24$, turnaround time for $P_2 = 27$ and turnaround time for $P_3 = 30$, so that the mean turnaround time = 27.
- While in SJF, the mean turnaround time = $(3+6+33)/3 = 14$.
- **Note:** Consider the case of four jobs, with run times of a , b , c , and d , respectively. The first job finishes at time a , the second finishes at time $a + b$, and so on. Thus, the mean turnaround time is $(4a + 3b + 2c + d)/4$. It is clear that a contributes more to the **average turnaround time** than the other times, so it should be the shortest job, with b next, then c , and finally d as the longest as it affects only its own **turnaround time**. The same argument applies equally well to any number of jobs.

Counter Example

- SJF is only optimal when all the jobs are available simultaneously.
- Example: Consider five jobs A, B, C, D and E with run times of 2, 4, 1, 1, and 1 respectively. Let their arrival times be 0, 0, 3, 3 and 3. Initially only A or B can be chosen since other jobs have not arrived yet. Using SJF, we will run the jobs in the order A, B, C, D, E with an average waiting time of 4.6. However, running them in the order B, C, D, E, A has an average waiting time of 4.4.



Shortest Remaining Time Next (SRTN)

- A preemptive version of shortest job first is **shortest remaining time next**.
- With this algorithm, the scheduler always chooses the process whose remaining run time is the shortest. The run time has to be known in advance.
- Any time a new process enters the pool of processes to be scheduled, the scheduler compares the expected value for its remaining processing time with that of the process currently scheduled. If the new process's time is less, the currently scheduled process is preempted.

Merits:

- Low average waiting time than SJF.
- Useful in timesharing.

Demerits:

- Very high overhead than SJF.
- Requires additional computation.
- Favors short jobs, long jobs can be victims of *starvation*.

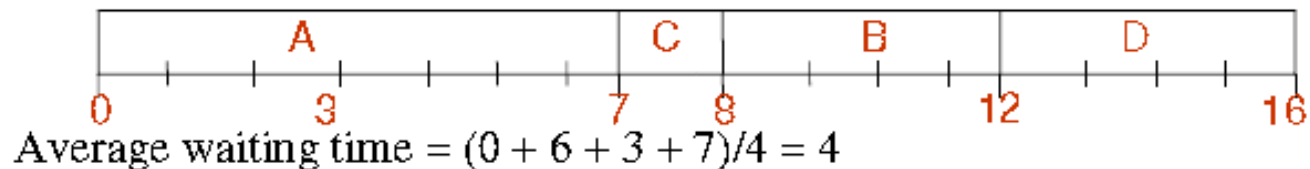
An example

SRT_N-Performance

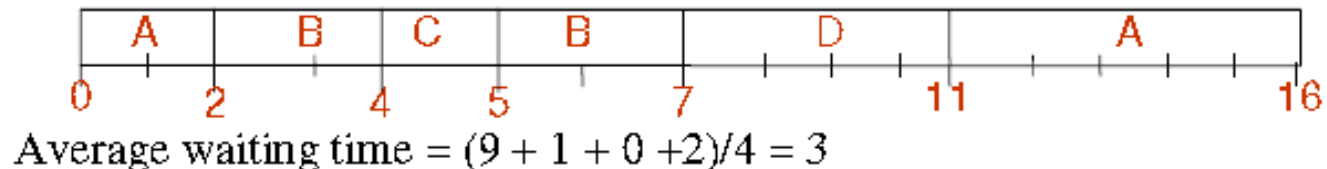
Scenario: Consider the following four processes with the length of CPU-burst time given in milliseconds:

Processes	Arrival Time	Burst Time
A	0.0	7
B	2.0	4
C	4.0	1
D	5.0	4

SJF:



SRT_N:



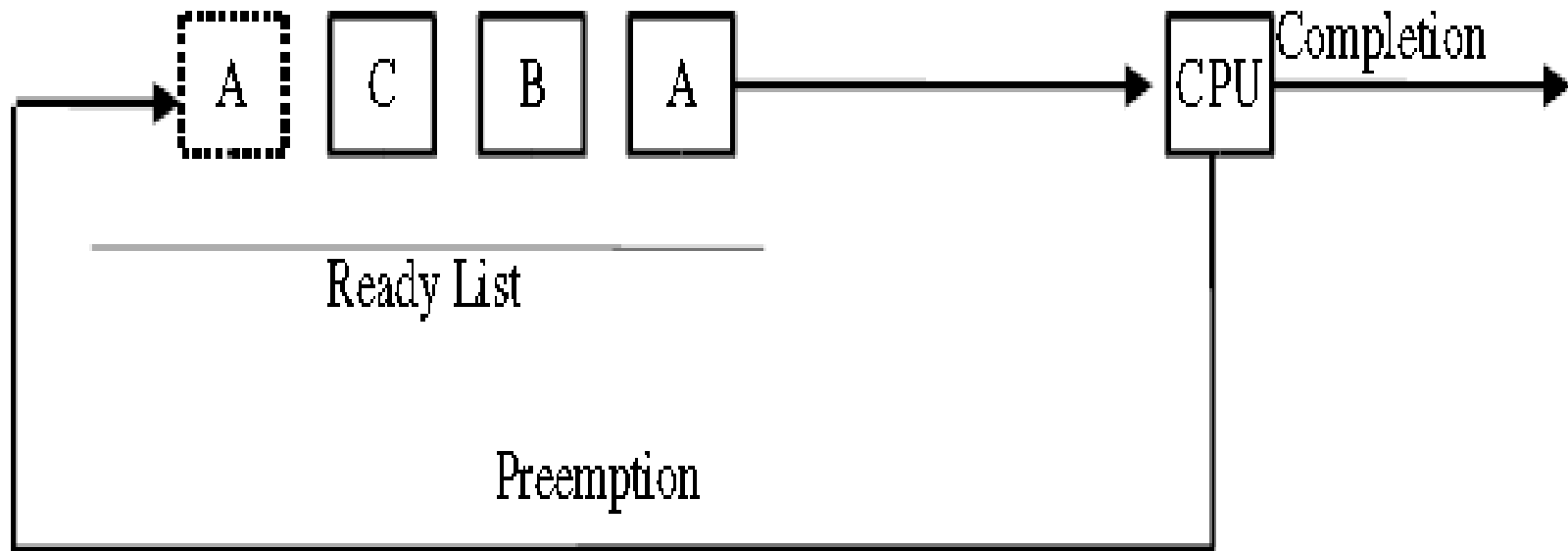
Round-Robin Scheduling

- Preemptive FCFS.
- Each process is assigned a time interval (**quantum**).
- After the specified quantum, the running process is preempted and a new process is allowed to run.
- Preempted process is placed at the back of the ready list.

Advantages:

- Fair allocation of CPU across the processes.
- Used in timesharing system.
- Low average waiting time when process lengths vary widely.

RR Scheduling...



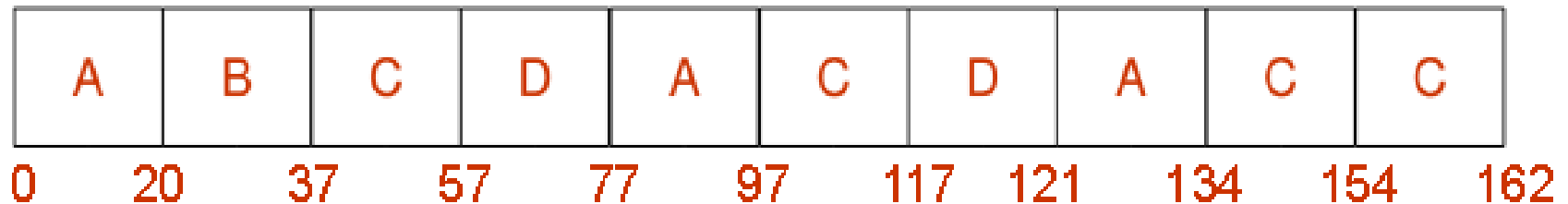
RR Performance

- Gives poor average waiting time when process lengths are identical.
Example: Let there be 10 processes each having 10 msec burst time and 1msec quantum. Now, performing RR-scheduling, all processes will complete after about 100 times. Clearly, FCFS is better in this case since there is about 20% time wastage in context-switching between processes.
- **Major performance factor: QUANTUM SIZE**
 - If the quantum size is very large, each process is given as much time as needs for completion; RR degenerates to FCFS policy.
 - If quantum size is very small, system gets busy at just switching from one process to another process; the overhead of context-switching degrades the system efficiency.
- **Optimal quantum size:** 80% of the CPU bursts should be shorter than the quantum. (20-50 msec reasonable for many general processes.)

Example of RR with Quantum = 20

Process	Burst Time
A	53
B	17
C	68
D	24

The Gantt chart is:



TU Exam Question 2068

- Round-robin scheduling behaves differently depending on its time quantum. Can the time quantum be set to make round robin behave the same as any of the following algorithms? If so how? Proof the assertion with an example.

(a) FCFS

(b) SJF

(c) SRTN

Priority Scheduling

- **Basic idea:** Each process is assigned a priority value, and runnable process with the highest priority is allowed to run.
- FCFS or RR can be used in case of tie.
- To prevent high-priority processes from running indefinitely, the scheduler may decrease the priority of the currently running process at each clock interrupt. If this action causes its priority to drop below that of the next highest process, a process switch occurs.

Priority Scheduling...

- Assigning priority: **static** and **dynamic**

Static: Some processes are more important than others. For example, a daemon process sending e-mail in the background is of lower importance than a process displaying a video film on the screen in real time. So some processes are assigned higher priority than others by user. **Problem: starvation.**

- **Dynamic:** Priority is assigned by the system to achieve certain system goals. For example, some processes are highly I/O bound and spend most of their time waiting for I/O to complete. Whenever such a process wants the CPU, it should be given the CPU immediately, to let it start its next I/O request. So the key idea here is to **decrease priority of CPU-bound processes and increase priority of I/O-bound processes.**

Many different policies are possible:

(1) Set the priority to $1/f$, where f is the fraction of the last quantum used by the process. Thus a process that used only 1 msec of its 50 msec quantum (heavily I/O bound) would get priority 50, while a process that ran 25 msec before blocking would get priority 2, and a process that used the whole quantum would get priority 1.

(2) $\text{priority} = (\text{time waiting} + \text{processing time}) / \text{processing time}$

Multiple Queues

- In this system, there are multiple queues with a different priority level set for each queue.
- Processes in the highest priority level use less CPU time than processes in the next-highest priority level, i.e., processes in the highest priority level are assigned small quanta than processes in the next-highest priority level.
- Processes may move between the queues. If a process uses its entire quantum, it will be moved to the tail of the next-lower-priority-level queue while if the process blocks before using its entire quantum it is moved to the next-higher-level-priority queue.
- **Advantage:** Better response for I/O-bound and interactive processes as they are set in the higher level priority queue.

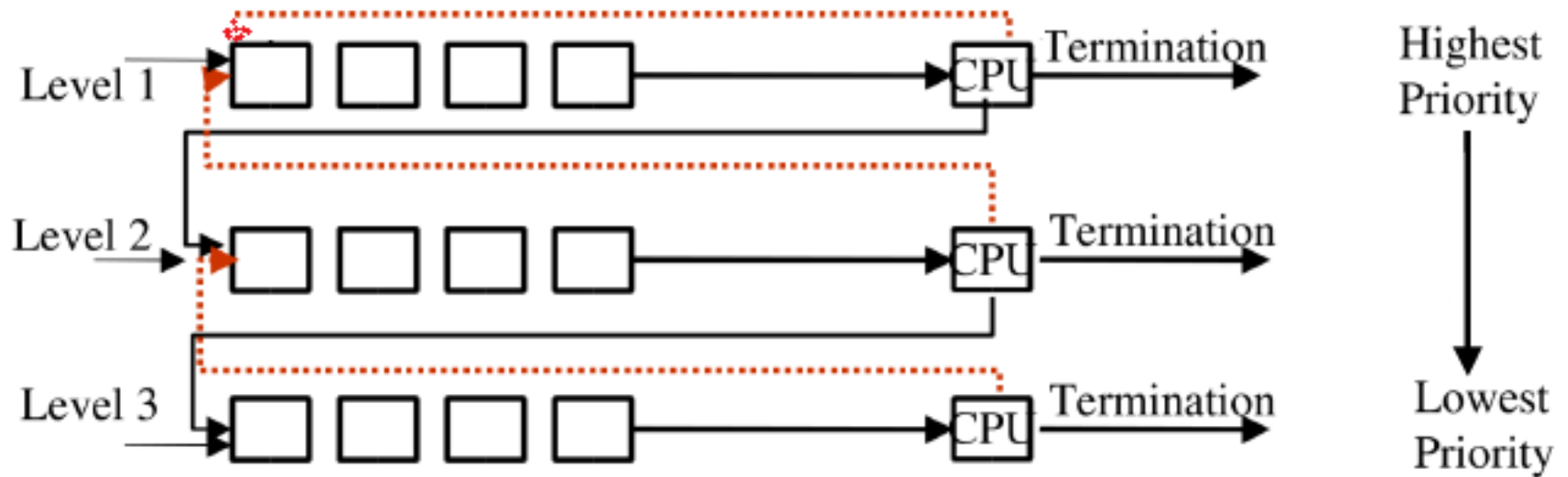


Fig: Multiple Queues

- **Example:** Consider a multiple queue with three queues numbered 1 to 3 with quantum size 8, 16 and 32 msec respectively. The scheduler first executes all processes in queue 1. Only when queue 1 is empty, it executes all processes in queue 2 and only when queue 1 and queue 2 are both empty, it executes processes in queue 3.

A process first enters in queue 1 and executes for 8 msec. If it does not finish, it moves to the tail of queue 2. If queue 1 is empty, the processes of queue 2 start to execute in FCFS manner with 16 msec quantum. If it still does not complete, it is preempted and moved to the tail of queue 3. If the process blocks before using its entire quantum, it is moved to the next higher level queue.

Classwork

For these processes listed in following table, draw a Gantt chart illustrating their execution using:

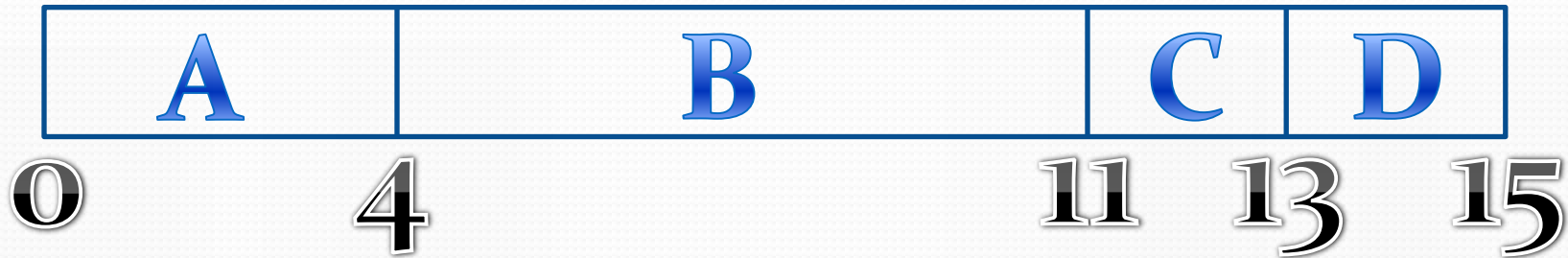
- (a) First-Come-First-Serve.
- (b) Short-Job-First.
- (c) Shortest-Remaining-Time-Next.
- (d) Round-Robin (quantum = 2).
- (e) Round-Robin (quantum = 1).

Process	Arrival Time	Burst Time
A	0.00	4
B	2.01	7
C	3.01	2
D	3.02	2

- i) What is the average turnaround time?
- ii) What is the average waiting time?

[Hint: turnaround time = finishing time – arrival time]

(a) FCFS



Average Waiting Time = $(0+1.99+7.99+9.98)/4 = 4.99$

Average Turnaround Time = $(4+8.99+9.99+11.98)/4 = 8.74$

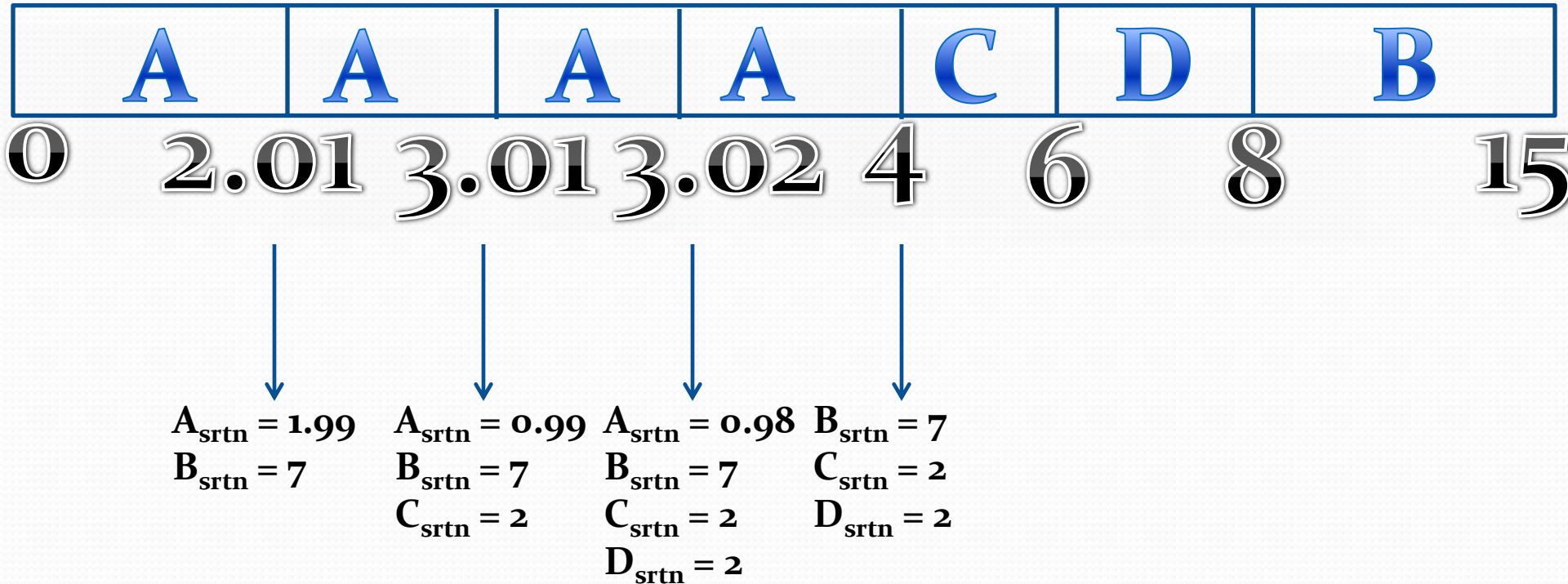
(b) SJF



$$\text{Average Waiting Time} = (0 + 0.99 + 2.98 + 5.99) / 4 = 2.49$$

$$\text{Average Turnaround Time} = (4 + 2.99 + 4.98 + 12.99) / 4 = 6.24$$

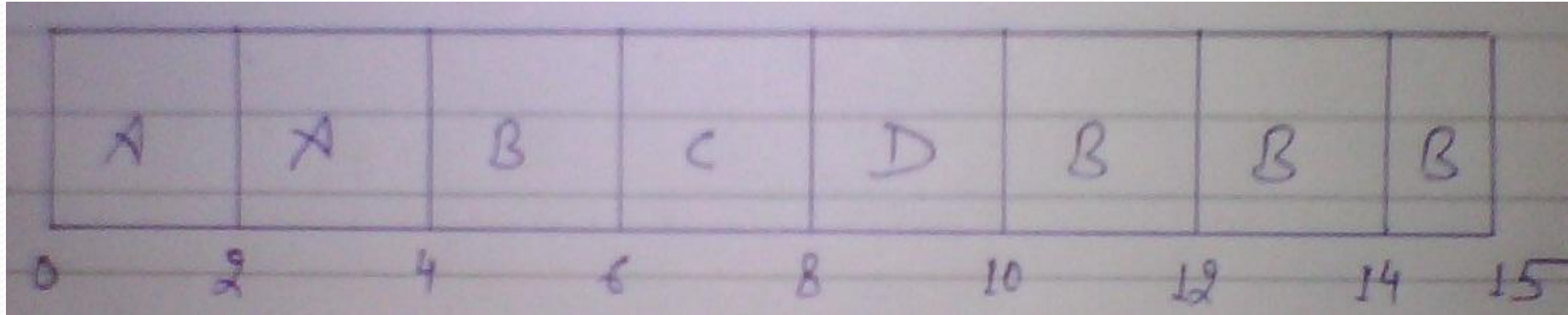
(c) SRTN



$$\text{Average Waiting Time} = (0 + 0.99 + 2.98 + 5.99) / 4 = 2.49$$

$$\text{Average Turnaround Time} = (4 + 2.99 + 4.98 + 12.99) / 4 = 6.24$$

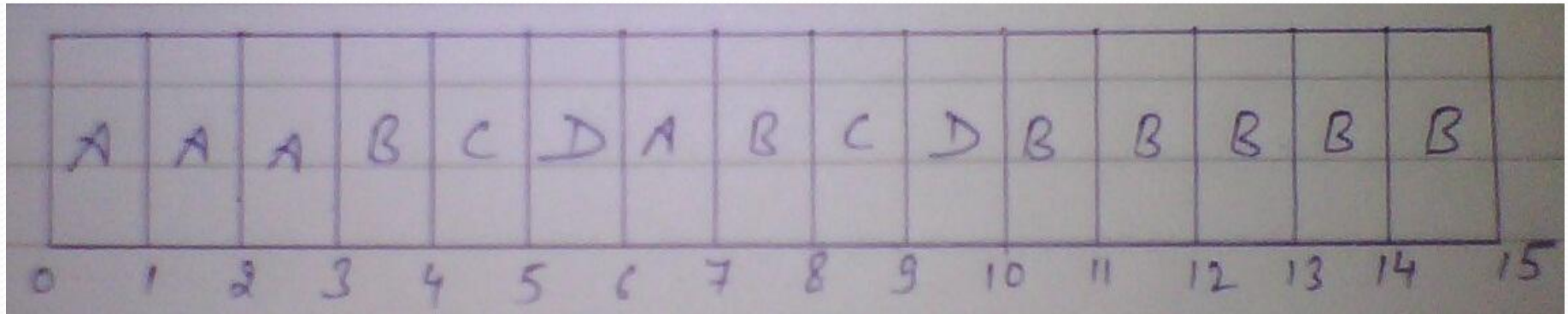
(d) RR (quantum = 2)



$$\text{Average Waiting Time} = (0 + (1.99 + 4) + 2.99 + 4.98) / 4 = 3.49$$

$$\text{Average Turnaround Time} = (4 + 12.99 + 4.99 + 6.98) / 4 = 7.24$$

(e) RR (quantum = 1)



Average Waiting Time = $((0+3)+(0.99+3+2)+(0.99+3)+(1.98+3))/4 = 4.49$

Average Turnaround Time = $(7+12.99+5.99+6.98)/4 = 8.24$

Homework

Consider the following set of processes, with the length of CPU-burst time given in milliseconds:

Process	Burst time	Priority
P ₁	10	3
P ₂	1	1
P ₃	2	3
P ₄	1	4
P ₅	5	2

The processes are assumed to have in the order P₁, P₂, P₃, P₄, P₅, all at time 0.

- Draw four Gantt charts illustrating the execution of these processes using FCFS, SJF, a non-preemptive priority (a smaller priority number implies a higher priority), and RR (quantum = 1) scheduling.
- What is the **turnaround time** of each process for each of the scheduling algorithms in part a?
- What is **waiting time** of each process for each of the scheduling algorithms in part a?
- Which of the schedules in part a results in the **minimal average waiting time**?

a. The four Gantt charts are

1					2	3	4	5					FCFS		
1	2	3	4	5	1	3	5	1	5	1	5	1	5	1	RR
2	4	3	5					1							SJF
2	5				1							3	4	Priority	

b. Turnaround time

	FCFS	RR	SJF	Priority
P_1	10	19	19	16
P_2	11	2	1	1
P_3	13	7	4	18
P_4	14	4	2	19
P_5	19	14	9	6

c. Waiting time (turnaround time minus burst time)

	FCFS	RR	SJF	Priority
P_1	0	9	9	6
P_2	10	1	0	0
P_3	11	5	2	16
P_4	13	3	1	18
P_5	14	9	4	1

d. Shortest Job First